Deployment Documentation

AI-Based Web Scraping Application

---

1. Introduction

With the rapid growth of data-driven applications, automated data extraction has become an essential requirement for modern systems. This project focuses on the deployment of an AI-based Web Scraping Application that collects data from multiple web sources, processes it intelligently, and serves the processed output through a web-based interface.
The deployment is carried out on a Virtual Private Server (VPS) using Nginx as a reverse proxy to ensure reliability, scalability, and secure public access.

2. Objective of Deployment

The primary objectives of deploying this application are:

- To host the AI-powered web scraping system in a production environment
- To make the application publicly accessible via IP/domain
- To ensure stable performance using a dedicated server
- To implement a secure and scalable server architecture

3. System Architecture Overview

The deployed system follows a client–server architecture:

- The client sends HTTP requests through a browser or API client.
- Nginx acts as a reverse proxy, handling incoming requests.
- Gunicorn manages multiple worker processes for the Python application.
- The backend application performs web scraping and AI-based processing.
- The processed data is returned to the client as a response.

This architecture improves performance, load handling, and security.

4. Technology Stack Used

Component	Technology
Backend Language	Python
Framework	Flask / FastAPI
Web Scraping	BeautifulSoup, Requests /
AI Processing	Rule-based / ML-based logic
Server	Windows VPS
Web Server	Nginx
Application Server	Gunicorn
Version Control	Git/Github

5. VPS Environment Setup

A Virtual Private Server (VPS) provides a dedicated and isolated environment for deploying applications.

5.1 VPS Access

The VPS is accessed securely using SSH (Secure Shell):

ssh root@<VPS_IP_ADDRESS>

5.2 System Update

The server packages are updated to ensure stability and security:

sudo apt update && sudo apt upgrade -y

6. Installation of Required Dependencies

6.1 Python Environment Setup

Python and virtual environments are installed to isolate project dependencies:

sudo apt install python3 python3-pip python3-venv -y

6.2 Nginx Installation

Nginx is installed to manage incoming HTTP requests:

sudo apt install nginx -y

6.3 Git Installation

Git is used to manage and deploy source code:

sudo apt install git -y

7. Project Deployment on VPS

7.1 Cloning the Project Repository

The project source code is cloned into the server:

cd /var/www

git clone <PROJECT_REPOSITORY_URL>

cd <PROJECT_FOLDER_NAME>

7.2 Virtual Environment Creation

python3 -m venv venv

source venv/bin/activate

7.3 Dependency Installation

pip install -r requirements.txt

8. Application Execution and Testing

Before production deployment, the application is tested locally on the VPS:

python app.py

or

uvicorn main:app --host 0.0.0.0 --port 8000

This step ensures that the application is functioning correctly.

9. Production Server Configuration using Gunicorn

Gunicorn is used as a **WSGI application server** for running Python applications in production.

pip install gunicorn

Application is started using:

gunicorn --bind 0.0.0.0:8000 app:app

Gunicorn enables better performance and process management.

10. Nginx Reverse Proxy Configuration

Nginx acts as an interface between users and the backend application.

10.1 Configuration File Creation

sudo nano /etc/nginx/sites-available/ai_scraping_project

10.2 Configuration Details

```
server {
    listen 80;
    server_name <DOMAIN_NAME or VPS_IP>;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

This configuration forwards incoming requests to the backend application securely.

10.3 Enabling and Restarting Nginx

sudo ln -s /etc/nginx/sites-available/ai_scraping_project /etc/nginx/sites-enabled

sudo nginx -t

sudo systemctl restart nginx

11. Firewall and Security Configuration

Firewall rules are applied to allow required traffic:

sudo ufw allow OpenSSH

sudo ufw allow 'Nginx Full'

sudo ufw enable

12. Deployment Validation and Testing

The deployment is validated by:

- Accessing the application via browser
- Testing API endpoints
- Verifying web scraping results
- Monitoring logs for errors

sudo systemctl status nginx

journalctl -u nginx

13. Challenges Faced and Resolution

- **Port conflicts:** Resolved by proper port binding
- **Permission issues:** Fixed using correct directory ownership
- **Nginx errors:** Debugged using Nginx logs and configuration testing

14. Advantages of VPS-Based Deployment

- Dedicated resources and better performance
- Improved security and control
- Scalability for future expansion
- Professional production-level deployment

15. Final Deployment URL

- After successful deployment and configuration, the AI-based Web Scraping application was made live and is accessible through the following URL:

  Deployed Application (API Documentation):

  https://projectai.arinova.studio:7080/docs

- This link provides access to the live deployed application, including the API documentation interface, where the endpoints of the AI-powered web scraping system can be tested and verified.

15. Conclusion

This documentation presents a complete deployment workflow of an AI-based Web Scraping Application using VPS and Nginx. The deployment ensures high availability, efficient request handling, and secure public access, making the application suitable for real-world and enterprise use.